Data Structures (CSN12101)
ASSIGNMENT-5

Name : Aayansh Niranjan Date :16/02/2026
Reg_No.: 20253004

Q1. Design, Develop and Implement a menu driven Program in C for the following operations on Doubly Linked List (DLL) of Employee Data with the fields: Employee id, Name, Dept, Designation, Sal, PhNo

a. Create a DLL of N Employees Data by using end insertion.
b. Display the status of DLL and count the number of nodes in it
c. Perform Insertion at End of DLL.
d. Perform Deletion at End of DLL
e. Perform Insertion at Front of DLL.
f. Perform Deletion at Front of DLL
g. Perform Insertion in DLL after a given Employee id Details.
h. Perform the deletion of particular employee after the given Employee id Details.
i. Exit
Find the Employee Details having maximum salary.
Find the Employee Details having minimum salary.

A1. CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct Node{
  int id,salary;
  long long phno;
  char name[100];
  char dept[100];
  char designation[100];
  struct Node* next;
  struct Node* prev;
};

struct Node* insert_end(struct Node* head){
  struct Node* temp=(struct Node*)malloc(sizeof(struct Node));
  temp->next=NULL;
  temp->prev=NULL;
  printf("Enter the ID : ");
  scanf("%d",&(temp->id));
  printf("Enter the salary : ");
  scanf("%d",&(temp->salary));
  printf("Enter the phone no. : ");
  scanf("%lld",&(temp->phno));
  printf("Enter the name : ");
  scanf("%s",temp->name);
  printf("Enter the department : ");
  scanf("%s",temp->dept);
  printf("Enter the designation : ");
  scanf("%s",temp->designation);
  if(head==NULL){return temp;}
  struct Node* mover=head;
  while(mover->next) mover=mover->next;
  mover->next=temp;
  temp->prev=mover;
  return head;
}
```

```
struct Node* insert_front(struct Node* head){
  struct Node* temp=(struct Node*)malloc(sizeof(struct Node));
  temp->next=head;
  temp->prev=NULL;
  printf("Enter the ID : ");
  scanf("%d",&(temp->id));
  printf("Enter the salary : ");
  scanf("%d",&(temp->salary));
  printf("Enter the phone no. : ");
  scanf("%lld",&(temp->phno));
  printf("Enter the name : ");
  scanf("%s",temp->name);
  printf("Enter the department : ");
  scanf("%s",temp->dept);
  printf("Enter the designation : ");
  scanf("%s",temp->designation);
  if(head!=NULL) head->prev=temp;
  return temp;
}
struct Node* insert_at_x(struct Node* head){
    struct Node* temp=(struct Node*)malloc(sizeof(struct Node));
    printf("Enter the id of the employee next to which you want to insert : ");
    int x;scanf("%d",&x);
    struct Node* mover=head;
    while(mover!=NULL && mover->id!=x) mover=mover->next;
    if(mover==NULL){
        printf("The required value was not found\n");
        free(temp);
        return head;
    }
    printf("Enter the ID : ");
    scanf("%d",&(temp->id));
    printf("Enter the salary : ");
    scanf("%d",&(temp->salary));
    printf("Enter the phone no. : ");
    scanf("%lld",&(temp->phno));
    printf("Enter the name : ");
    scanf("%s",temp->name);
    printf("Enter the department : ");
    scanf("%s",temp->dept);
    printf("Enter the designation : ");
    scanf("%s",temp->designation);
    temp->next = mover->next;
    temp->prev = mover;
    if(mover->next != NULL) mover->next->prev=temp;
    mover->next=temp;
    return head;
}
struct Node* delete_end(struct Node* head){
    if(head==NULL) return head;
    struct Node* temp=head;
    if(temp->next==NULL){
        free(temp);
        return NULL;
    }
    while(temp->next!=NULL)temp=temp->next;
    temp->prev->next=NULL;
    free(temp);
    return head;
}
```

```
struct Node* delete_front(struct Node* head){
    if(head==NULL)return NULL;
    if(head->next==NULL){free(head);return NULL;}
    struct Node* temp=head->next;
    temp->prev=NULL;
    free(head);
    return temp;
}

struct Node* delete_after_id(struct Node* head){
    if(head==NULL) return head;
    int x;scanf("%d",&x);
    struct Node* mover=head;
    while(mover!=NULL && mover->id!=x) mover=mover->next;
    if(mover==NULL || mover->next==NULL) return head;
    struct Node* temp=mover->next;
    mover->next=temp->next;
    if(temp->next!=NULL) temp->next->prev=mover;
    free(temp);
    return head;
}

void printDLL(struct Node* head){
    if(head == NULL){
        printf("List is empty\n");
        return;
    }
    struct Node* temp=head;
    printf("Employee Details:\n");
    while(temp!=NULL){
        printf("ID : %d\n",temp->id);
        printf("Salary : %d\n",temp->salary);
        printf("Phone : %lld\n",temp->phno);
        printf("Name : %s\n",temp->name);
        printf("Department : %s\n",temp->dept);
        printf("Designation : %s\n",temp->designation);
        temp=temp->next;
    }
}

void find_max_salary(struct Node* head){
    if(head==NULL){
        printf("List is empty\n");
        return;
    }
    struct Node* max=head;
    struct Node* temp=head->next;
    while(temp!=NULL){
        if(temp->salary > max->salary) max=temp;
        temp=temp->next;
    }
    printf("\nEmployee with Maximum Salary:\n");
    printf("ID : %d\n",max->id);
    printf("Salary : %d\n",max->salary);
    printf("Phone : %lld\n",max->phno);
    printf("Name : %s\n",max->name);
    printf("Department : %s\n",max->dept);
    printf("Designation : %s\n",max->designation);
}
```

```
void find_min_salary(struct Node* head){
    if(head == NULL){
        printf("List is empty\n");
        return;
    }
    struct Node* min=head;
    struct Node* temp=head->next;
    while(temp != NULL){
        if(temp->salary < min->salary)min=temp;
        temp=temp->next;
    }
    printf("\nEmployee with Minimum Salary:\n");
    printf("ID : %d\n",min->id);
    printf("Salary : %d\n",min->salary);
    printf("Phone : %lld\n",min->phno);
    printf("Name : %s\n",min->name);
    printf("Department : %s\n",min->dept);
    printf("Designation : %s\n",min->designation);
}

int main(){
    struct Node* head=NULL;
    int choice;
    while(1){
        printf("\nSelect your choice : \n");
        printf("1 : Insert employee at end\n");
        printf("2 : Delete employee from end\n");
        printf("3 : Insert employee at front\n");
        printf("4 : Delete employee from front\n");
        printf("5 : Insert employee after given ID\n");
        printf("6 : Delete employee after given ID\n");
        printf("7 : Print the DLL\n");
        printf("8 : Find the maximum salary\n");
        printf("9 : Find the minimum salary\n");
        printf("10 : Exit\n");
        printf("Your Input : ");
        scanf("%d", &choice);
        switch(choice){
            case 1:
                head = insert_end(head);
                break;
            case 2:
                head = delete_end(head);
                break;
            case 3:
                head = insert_front(head);
                break;
            case 4:
                head = delete_front(head);
                break;
            case 5:
                head = insert_at_x(head);
                break;
            case 6:
                printf("Enter ID after which to delete: ");
                head = delete_after_id(head);
                break;
            case 7:
                printDLL(head);
                break;
            case 8:
```

```
            case 8:
                find_max_salary(head);
                break;
            case 9:
                find_min_salary(head);
                break;
            case 10:
                printf("Exiting program...\n");
                return 0;
            default:
                printf("Invalid choice. Try again.\n");
        }
    }
    return 0;
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_5$ ./p1

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 1
Enter the ID : 101
Enter the salary : 10000
Enter the phone no. : 8090895438
Enter the name : Aayansh
Enter the department : HR
Enter the designation : manager

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 1
Enter the ID : 102
Enter the salary : 18000
Enter the phone no. : 8194269643
Enter the name : Name2
Enter the department : Tech
Enter the designation : CTO
```

```
Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 1
Enter the ID : 103
Enter the salary : 12000
Enter the phone no. : 8327486239
Enter the name : Name3
Enter the department : Operations
Enter the designation : Worker

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 7
Employee Details:
ID : 101
Salary : 10000
Phone : 8090895438
Name : Aayansh
Department : HR
Designation : manager
ID : 102
Salary : 18000
Phone : 8194269643
Name : Name2
Department : Tech
Designation : CTO
```

```
ID : 103
Salary : 12000
Phone : 8327486239
Name : Name3
Department : Operations
Designation : Worker

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 2

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 4

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input : 8

Employee with Maximum Salary:
ID : 102
Salary : 18000
Phone : 8194269643
Name : Name2
Department : Tech
Designation : CTO

Select your choice :
1 : Insert employee at end
2 : Delete employee from end
3 : Insert employee at front
4 : Delete employee from front
5 : Insert employee after given ID
6 : Delete employee after given ID
7 : Print the DLL
8 : Find the maximum salary
9 : Find the minimum salary
10 : Exit
Your Input :
```

Write a C Program to Demonstrate Circular linked list.

Creation of Circular linked list
Insertion at beginning
Insertion at remaining
Deletion at beginning
traverse
Search
sort
update
Exit

A2. CODE:

```
#include<stdio.h>
#include<stdlib.h>

struct Node{
    int data;
    struct Node* next;
};
struct Node* head=NULL;
void create(int val){
    struct Node* n=malloc(sizeof(struct Node));
    n->data=val;
    if(head==NULL){head=n;n->next=head;return;}
    struct Node* t=head;
    while(t->next!=head)t=t->next;
    t->next=n;
    n->next=head;
}
void insertBeg(int val){
    struct Node* n=malloc(sizeof(struct Node));
    n->data=val;
    if(head==NULL){head=n;n->next=head;return;}
    struct Node* t=head;
    while(t->next!=head)t=t->next;
    n->next=head;
    t->next=n;
    head=n;
}
void insertEnd(int val){create(val);}
void deleteBeg(){
    if(head==NULL)return;
    if(head->next==head){free(head);head=NULL;return;}
    struct Node* t=head,*last=head;
    while(last->next!=head)last=last->next;
    head=head->next;
    last->next=head;
    free(t);
}
void traverse(){
    if(head==NULL)return;
    struct Node* t=head;
    do{printf("%d ",t->data);
      t=t->next;}
    while(t!=head);
    printf("\n");
}
```

```
void search(int key){
   if(head==NULL)return;
   struct Node* t=head;int pos=1;
   do{
      if(t->data==key){printf("Found at %d\n",pos);return;}
      t=t->next;pos++;
   }while(t!=head);
   printf("Not Found\n");
}

void sort(){
   if(head==NULL)return;
   struct Node *i,*j;int temp;
   for(i=head;i->next!=head;i=i->next)
      for(j=i->next;j!=head;j=j->next)
         if(i->data>j->data){
           temp=i->data;
           i->data=j->data;
           j->data=temp;
         }
}

void update(int old,int newv){
   if(head==NULL)return;
   struct Node* t=head;
   do{
      if(t->data==old){t->data=newv;return;}
      t=t->next;
   }while(t!=head);
}

int main(){
   int ch,val,old,newv;
   while(1){
      printf("\n1.Create\n2.InsertBeg\n3.InsertEnd\n4.DeleteBeg\n5.Traverse\n6.Search\n7.Sort\
n8.Update\n9.Exit\nChoice:");
      scanf("%d",&ch);
      switch(ch){
         case 1:printf("Val:");scanf("%d",&val);create(val);break;
         case 2:printf("Val:");scanf("%d",&val);insertBeg(val);break;
         case 3:printf("Val:");scanf("%d",&val);insertEnd(val);break;
         case 4:deleteBeg();break;
         case 5:traverse();break;
         case 6:printf("Search:");scanf("%d",&val);search(val);break;
         case 7:sort();break;
         case 8:printf("Old New:");scanf("%d%d",&old,&newv);update(old,newv);break;
         case 9:return 0;
         default:printf("Invalid\n");
      }
   }
}
```

OUTPUT:

```
1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:1
Val:10

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:3
Val:11

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:2
Val:4
```

```
1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:5
4 10 11

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:6
Search:11
Found at 3

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:2
Val:16
```

```
1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:7

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:5
4 10 11 16

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:8
Old New:4
6
```

```
1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:5
6 10 11 16

1.Create
2.InsertBeg
3.InsertEnd
4.DeleteBeg
5.Traverse
6.Search
7.Sort
8.Update
9.Exit
Choice:9
```